

A lightweight and robust approach to IoT intrusion detection based on ensemble deep learning

Huiyao Dong & Igor Kotenko

To cite this article: Huiyao Dong & Igor Kotenko (02 Feb 2026): A lightweight and robust approach to IoT intrusion detection based on ensemble deep learning, International Journal of Parallel, Emergent and Distributed Systems, DOI: [10.1080/17445760.2026.2624480](https://doi.org/10.1080/17445760.2026.2624480)

To link to this article: <https://doi.org/10.1080/17445760.2026.2624480>



Published online: 02 Feb 2026.



Submit your article to this journal [↗](#)



Article views: 100



View related articles [↗](#)



View Crossmark data [↗](#)



A lightweight and robust approach to IoT intrusion detection based on ensemble deep learning

Huiyao Dong ^a and Igor Kotenko ^b

^aCenter for Intelligent Secure Systems, King Fahd University of Petroleum & Minerals, Dhahran, Saudi Arabia;

^bSt. Petersburg Federal Research Center of the Russian Academy of Sciences, St. Petersburg, Russia

ABSTRACT

The inherent vulnerabilities and distributed nature of IoT systems necessitate IDS that balance accuracy with computational efficiency. This study proposes a lightweight, distributed IDS leveraging ensemble deep learning to address IoT security challenges. By integrating multiple lightweight models (DNN, CNN, GRU) via various ensemble strategies, the framework enhances detection capabilities while minimizing resource overhead. Evaluated on the CICIDS2017 and Edge-IIoT datasets, the ensemble approach outperforms single-model solutions, specifically stacking and boosting in rare threat detection. This work demonstrates the viability of decentralized, resource-aware IDS deployments, offering a scalable solution for securing dynamic IoT ecosystems amidst evolving cyber threats.

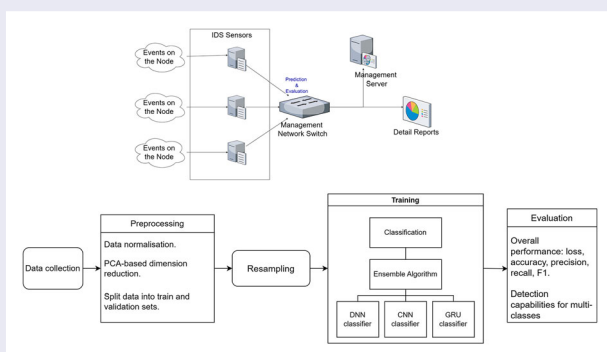
ARTICLE HISTORY

Received 5 March 2025

Accepted 26 January 2026

KEYWORDS

Internet of Things (IoT); ensemble learning; deep learning cyberattacks; intrusion detection



1. Introduction

The Internet of Things (IoT) has experienced unprecedented growth, with the number of connected devices expected to increase by 13% to 18.8 billion by the end of 2024. This trend is projected to continue, with over 40 billion IoT devices anticipated by 2030, driven by advancements in smart homes, industrial automation, and healthcare [1]. This exponential growth, however, exacerbates existing security vulnerabilities, as the sheer scale of interconnected devices creates a larger attack surface for malicious actors. The IoT ecosystem

CONTACT Igor Kotenko  ivkote@comsec.spb.ru, ivkote1@mail.ru  St. Petersburg Federal Research Center of the Russian Academy of Sciences, St. Petersburg 199178, Russia

is divided into three critical layers – user control terminal, cloud service terminal, and IoT terminal – each vulnerable to distinct threats. For instance, the user control terminal faces issues such as code protection flaws, operational security threats, and terminal authority management lapses, while the cloud service terminal is exposed to traditional network layer vulnerabilities [2]. The attacks targeting IoT systems are typically complex due to the diverse nature of devices, their low power capabilities, and the ambiguity surrounding device ownership. The interconnected nodes and devices used for data storage and transmission also expose networks and systems to potential attacks.

To prevent those security threats, potentially compromising data storage and system functionality, intrusion detection systems (IDSs) are the first choice. Signature-based IDS solutions have proven effective against known intrusions but fail against sophisticated ones, while anomaly-based IDS involving machine learning (ML) and deep learning (DL) techniques can filter out unknown network intrusions. The deep, fully connected network, together with a characteristic function, could perform not only anomaly detection but multiple other classification tasks at the same time, indicating its robustness [3]. According to the authors' experience, part from deep models, utilizing simple binary classifiers like Decision Tree (DT) and K Nearest Neighbours (KNN) with boosting algorithms to build ensemble models is also a feasible solution, which enhanced the weak learners' ability to adapt to complex network traffic classification tasks [4, 5].

In addition to effectiveness, the recent research in the field also emphasizes the importance of timeliness and efficiency. Sanju [6] introduced an ensemble framework of recurrent neural networks to improve intrusion detection in IoT systems. This approach used long short-term memory and gated recurrent unit models, along with Harris-Hawk optimization and fractional derivative mutation for feature selection. Alem et al. [7] proposed an efficient autonomous IDS that combined specification-based and behavioral-based IDS. Privacy concerns were addressed through a federated learning-based anomaly detection mechanism [8]. Lastly, Bertoli et al. [9] utilized unsupervised learning models in an ensemble learning framework for a federated learning-based IDS, demonstrating strong generalisability across heterogeneous networks.

While DL models offer non-linear flexibility, they may encounter limitations such as high variance, inconsistent predictions, and overfitting resulting from overly complex learned representations. Integrating DL models with ensemble learning frameworks represents a significant advancement. Ensemble learning aggregates the predictions of multiple models to make optimal decisions, thereby improving model performance, generalization, and relevance in security applications. This strategy helps alleviate overfitting and enhances predictive accuracy on unseen data. Additionally, employing shallow and simple neural networks as weak classifiers within the ensemble DL framework can address individual model limitations, boosting robustness and performance by leveraging diverse architectures. Particularly for complex tasks like network intrusion detection or malware image analysis, which involve high-dimensional data and numerous features, the ensemble DL approach can be practical for learning essential patterns and relationships without necessitating excessively deep and intricate model structures. Building upon the integration of DL models and ensemble learning frameworks described above, it is essential to consider the distinct challenges and requirements posed by designing IDSs for distributed IoT environments. The importance of developing distributed IDS frameworks is paramount, as such a framework offers the advantage of decentralizing the detection process and allowing

real-time, local intrusion detection at the edge of the network, which aligns perfectly with the distributed nature of IoT networks. The distributed IDS is constructed based on the central monitoring node's big data processing system and hidden agents. Hence, a well-designed ensemble DL approach should be lightweight, adaptive, scalable, and able to function across multiple computing nodes in the IoT environment.

In this study, we conduct a comparative analysis of various ensemble DL learning strategies in the network intrusion domain. We implement three DL models with distinct structures: a fully connected deep neural network (DNN), a convolutional neural network (CNN), and a gated recurrent unit (GRU)-based network. Leveraging these weak learners, we deploy and evaluate several ensemble frameworks, including hard voting, soft voting, stacking, and boosting. Our experimental results, conducted on two publicly available datasets, demonstrate that the ensemble DL approach significantly enhances intrusion detection capabilities. The improved performance, attributed to the ensemble algorithm, also exhibits robust generalizability across diverse network environments. Take the experiment on CICIDS2017 [10] for example, the boosting algorithm strikes the highest accuracy of 98.31% and detection rate of 98.32%, while stacking can enhance the overall performance regardless of the existence of weak classifiers with underperformance. The lightweight nature of the DL models ensures that they can be deployed on resource-constrained IoT devices without overwhelming their processing and memory capabilities. Furthermore, the selected ensemble approaches are computationally efficient and allow for the combination of predictions from multiple base models, enhancing the overall accuracy and robustness. By distributing the computation and detection tasks across multiple IoT devices, the system can efficiently process network traffic and detect intrusions in a timely manner. This distributed ensemble learning-based IDS architecture enables scalability, as additional IoT devices can be easily integrated into the system to handle increasing network traffic loads. The system can also adapt to the dynamic nature of IoT environments by continuously learning and updating its models based on incoming network traffic. By leveraging the strengths of ensemble learning and distributed computing, this approach provides a promising solution against evolving cyber threats.

This work significantly extends our preliminary framework [11]. Building on that foundation, we refine the workflow, rigorously formalize the methodology, and expand the experimental validation with new datasets. The rest of the paper is as follows: In Section 2, we review the current research status in ensemble learning and ensemble DL. Section 3 demonstrates the proposed methodology. Section 4 presents the experiment results of individual weak learners and ensemble models. Lastly, we conclude the paper and discuss the future plan.

2. Related work

Ensemble learning methods have been extensively studied in the field of network security. A recent survey summarized various ensemble learning methods from different categories; homogeneous ensembles, like Random Forest (RF), excel in consistency, while heterogeneous ensembles, like stacking, offer flexibility in combining diverse models [12]. However, it also identified a gap in the comprehensive comparison of ensemble strategies such as voting, stacking, and boosting, particularly in IoT security contexts.

Many studies employing ensemble learning models prioritize the binary classification due to its simplicity and applicability to detecting malicious versus benign network traffic. In the research by Fitni et al. [13], ensemble learning models such as RF and Gradient Boosting (GB) not only surpassed individual ML models like DT and Logistic Regression (LR) but also outperformed the basic DNN model. Similarly, Abrami et al. [14] implemented a stacking classifier using Support Vector Machine (SVM), Naïve Bayes (NB), and RF as base classifiers, with LR as the meta-level classifier; the experiment demonstrated that the ensemble model outperformed all base classifiers. In contrast, Gao et al. [15] reported conflicting results in multi-label classification. While RF and Adaboost achieved competitive overall accuracy compared to a deep DNN model, these ensemble learning models notably lagged behind the DNN in detecting rare attacks. Furthermore, while ensemble learning models often outperform basic machine learning models, simple models can be used for feature selection or importance calculation since they are computationally economical, interpretable, and appropriate for identifying key features in high-dimensional datasets. As suggested by Din et al. [16], they used DT to calculate feature importance, resulting in improved performance of voting ensemble models.

Recently, researchers have realized that the decentralized or distributed IDS can be a feasible solution to the IoT environment, as each node in such a framework can handle data independently and collaborate with other nodes to enhance detection accuracy and threat response time. Moreover, this can reduce dependency on centralized storage and computation systems and enhance the fault tolerance of intrusion detection. For instance, blockchain's decentralized architecture, while secure, is susceptible to DDoS attacks targeting consensus mechanisms or transaction validation. Kumar et al. [17] addressed the vulnerability of blockchain to DDoS attacks and presented a novel distributed IDS using fog computing and several models to detect DDoS attacks in blockchain-enabled IoT networks, including RF and an optimized gradient tree boosting system (XGBoost) on distributed fog nodes. Another similar approach is an anomaly detection system decentralizing cloud-based security architecture to local fog nodes [18], where RF was chosen as the primary model for achieving the highest accuracy. These works demonstrate the practical value of distributed, ensemble learning-based intrusion detection approaches. Additionally, other ML techniques are viable as well. Sharma et al. [19] proposed a tree-based probabilistic distributed IDS to detect anomalies in service-to-service interactions across multiple edge clusters, specifically designed for the Zero Trust Network environment, which requires continuous verification and robust anomaly detection. Hnamte et al. [20] proposed a distributed detection system utilizing a hybrid DL model, incorporating CNN layers, bidirectional LSTM layers, and fully connected layers.

Ensemble DL methodologies present a promising avenue for enhancing network intrusion detection capabilities. Nonetheless, the predominant emphasis remains on optimizing the architecture of individual base learners, thereby overlooking the inherent advantages of exploring the ensemble framework itself. While stacking has demonstrated notable success in numerous studies, other ensemble strategies could potentially provide complementary or superior advantages, depending on the context. Thus, there exists a strong rationale for expanding research efforts to encompass a broader array of ensemble techniques, including soft voting and boosting. Consequently, this research endeavors to delve into the examination of diverse ensemble methods. Moreover, a distributed approach to IDSs necessitates efficient models with fewer parameters that can effectively handle the voluminous

and diverse data generated by IoT devices. While we examine different parameters to select the appropriate structures of the base models, we aim to ensure that the models are not over-parametrized in order to adapt to the processing capabilities across various nodes.

3. Methodology

3.1. Distributed IDS design

A distributed IDS comprises a network of multiple IDSs that share logs and alert information, enabling collaborative threat detection across heterogeneous IoT environments [21]. Given the distributed nature of IoT, where devices operate independently across diverse networks, a centralized IDS is often impractical due to latency and scalability issues. Distributed IDSs offer enhanced scalability, fault tolerance, and real-time threat detection, making them particularly suitable for IoT ecosystems. The type and volume of information shared among the IDSs are initially configured together and then fine-tuned adaptively on each node. Real-time network monitoring and detection are independently carried out on each node, employing lightweight algorithms to ensure rapid detection of suspicious activities without relying on centralized servers [10].

We propose the distributed IDS as depicted in Figure 1, reflecting the complexities of a heterogeneous IoT environment, which comprises three core components: (1) local nodes for real-time monitoring, (2) a central management server for detailed analysis, and (3) a communication protocol for seamless information exchange.

The main workflow involves network data processing, distributed monitoring, centralized evaluation, and classification reports, coupled with decentralized response generation strategies. Given the varied capabilities and protocols among IoT devices, when a traffic packet enters the system, it first arrives at one of the network nodes, equipped to handle diverse network conditions with preliminary filtering applications that also serve as

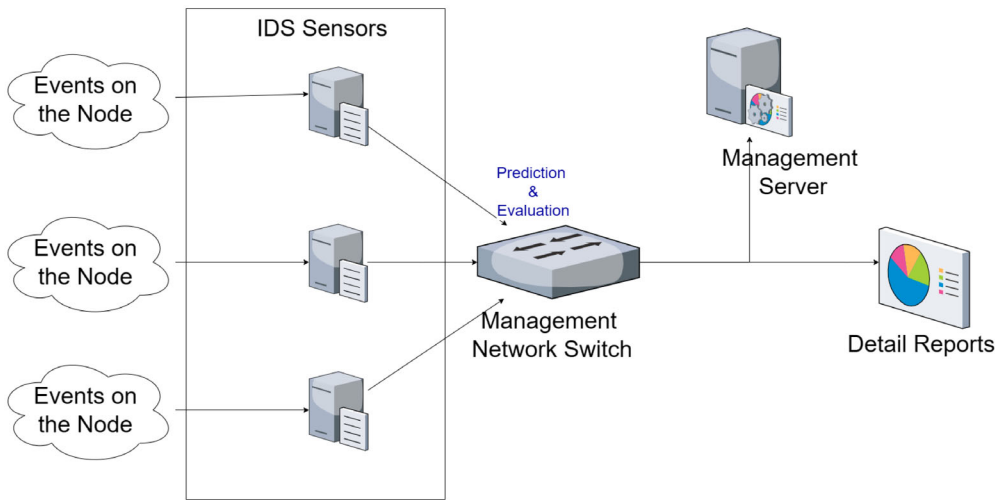


Figure 1. Distributed intrusion detection system architecture.

network traffic load dividers. Those applications, such as firewalls and packet analysers, perform initial traffic screening, reducing the load on local IDS by filtering out known benign traffic. The packet is subsequently analyzed by the local IDS on that node, leveraging decentralized control to enable autonomous decision-making. If identified as malicious by the ensemble DL model, the IDS generates an alert, integrating trust-enhancing mechanisms like blockchain for secure information exchange. Nodes share information about detected anomalies and updated detection policies, facilitated through a standardized communication protocol to ensure consistent threat detection across the heterogeneous distributed environment. The protocol includes synchronization mechanisms that periodically align detection policies across nodes. The management network switch, designed for reliability and resilience, aggregates data from all nodes for further centralized analysis and evaluation. Considering vendor and legal constraints, the management server updates detection policies, disseminating them back to the IDS nodes with strategies that respect operational diversity. The effectiveness of this system depends on the detection mode's independent capabilities, standardized protocol for seamless exchange, a robust management network switch, comprehensive reports with detailed traffic analysis, and the timely policy updates and dissemination to nodes, ensuring resilience and trust in response to threats. Meanwhile, achieving high detection accuracy frequently necessitates balancing computing resources, especially in resource-constrained IoT devices, where the balance of performance and efficiency is crucial.

3.2. Workflow

This sub-section focuses on the workflow of data-driven network traffic classification, as illustrated in Figure 2. The workflow begins with the data collection phase, where network traffic data is gathered from IoT networks and devices. This involves the aggregation of traffic traces and session logs, captured using network sensors and IoT device logs.

In the preprocessing stage, several essential procedures are applied to prepare the data for model training. Data normalization ensures that all features are scaled to a uniform range, preventing bias towards features with larger magnitudes. Principal Component Analysis (PCA) is employed for dimensionality reduction [22], selecting the most informative features and reducing computational overhead in resource-constrained IoT environments. Finally, the data is partitioned into training and validation sets to ensure that the model is evaluated on unseen data, improving generalizability and preventing overfitting. The

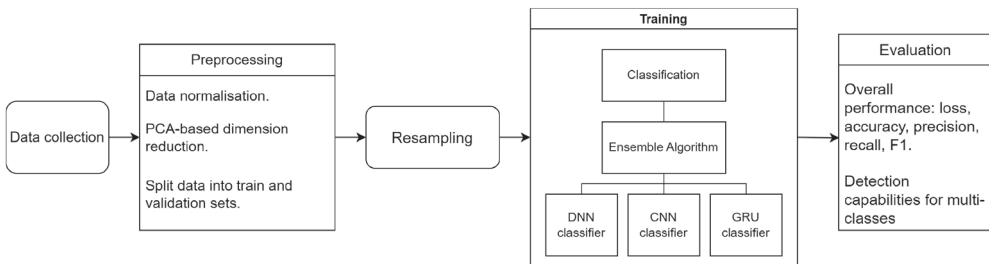


Figure 2. Workflow of the proposed approach.

data resampling stage addresses the issue of class imbalance, where benign traffic samples often dominate in IoT datasets. Random undersampling is applied to the majority class (benign traffic), while SMOTE (Synthetic Minority Oversampling Technique) [23] is used to increase the volume of minority samples by generating synthetic data based on the K-nearest neighbor rule.

In the model training stage, an ensemble model comprising three base classifiers – DNN-based, CNN-based, and GRU-based – is implemented. DNNs are chosen for their ability to model complex relationships, CNNs for their effectiveness in capturing spatial patterns, and GRUs for their ability to handle sequential data with temporal dependencies. By combining the outputs of these classifiers using a weighted averaging method, the ensemble approach mitigates the weaknesses of individual models, resulting in a more robust and accurate classification system. Finally, the model performance is evaluated using key metrics such as accuracy, precision, recall, and the F1 score. Accuracy measures overall correctness, precision evaluates the model's ability to avoid false positives, recall assesses its ability to detect true positives, and the F1 score provides a balanced measure of precision and recall. The detection ability of different network types, especially rare attacks, is also investigated, as these attacks often target specific vulnerabilities and can cause significant damage to the system. The details of the base classifiers and ensemble algorithms are introduced below.

3.3. Base classifier

In ensemble learning, the base classifiers independently perform prediction or classification tasks, and their results are combined to make a final prediction. As discussed in [12], the base classifiers should possess diverse learning approaches, accurate individual performance, and the scalability to handle massive data or numerous features. Diversity among base classifiers ensures that the ensemble can capture different aspects of the data, reducing the risk of correlated errors and improving overall robustness [24]. In addition to these characteristics, to optimize the model for adaptation to IoT environments, we aim to implement lightweight DL classifiers that ensure low resource occupancy and high efficiency. Given the limited computational resources and energy constraints of IoT devices, the models are designed with a minimal number of layers and parameters, while still maintaining high accuracy in detecting network threats. We implement three types of DL classifiers, as presented in Figure 3. The introduction of each model is listed below:

- (1) DNN serves as the foundational layer in DL models, employing numerous neurons within each layer to capture abstracted features from the received data, along with non-linear activation functions. DNN classifiers are versatile and capable of handling one-dimensional (1-D) sequential data. In this base classifier, we implement five fully connected layers with 128 neuron units, followed by a dropout layer to avoid overfitting. The dropout layer randomly deactivates 10% of neurons during training, preventing overfitting by encouraging the network to learn redundant representations of the data [25]. DNNs' ability to learn complex, high-dimensional patterns may explain their superior performance in detecting rare attacks, which often involve subtle, non-linear relationships. Its ability to model complex, non-linear relationships makes it particularly effective in detecting subtle patterns in network traffic, complementing the strengths of other models in the ensemble.

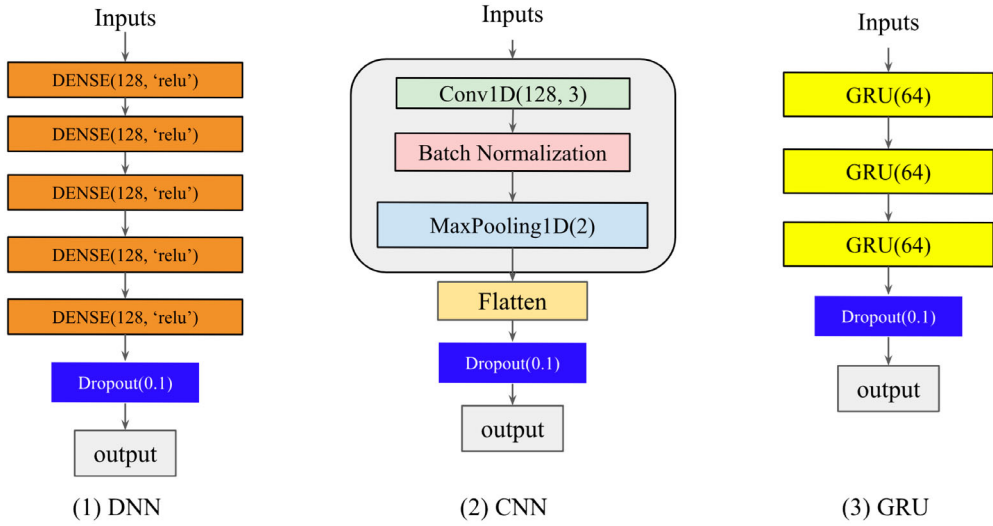


Figure 3. The structure of the three base classifiers.

- (2) CNN is specialized in processing grid-like data, such as time series (1-D) or images (2-D or 3-D). The convolutional layers in CNN utilize filters to capture spatial hierarchies. In this classifier, we implement three convolutional blocks, each including one convolutional layer with 128 units, one batch normalization layer to stabilize the data, and a MaxPooling layer for dimension reduction and denoising. Batch normalization stabilizes the learning process by normalizing the input of each layer, reducing internal covariate shift and accelerating training. MaxPooling reduces the dimensionality of the data by down-sampling, decreasing computational complexity while enhancing the model's ability to generalize by focusing on the most salient features. After the convolutional blocks, a flattening layer is applied to prepare the data for the final classification. The CNN's ability to capture spatial hierarchies in network traffic data makes it particularly effective in identifying localized patterns, such as anomalies in specific time intervals [26].
- (3) GRU represents an advanced version of RNN models, incorporating a simplified gate structure to address the vanishing gradient problems associated with the original RNN. The GRU employs reset and update gates to control the flow of information, allowing it to selectively retain or discard information from previous time steps, mitigating the vanishing gradient problem. Considering the computational complexity of such a layer, this base classifier contains only three GRU layers with 64 units, followed by a dropout layer to prevent overfitting. Its ability to model temporal dependencies in sequential data makes it particularly effective in detecting coordinated attacks that unfold over time, complementing the spatial strengths of the CNN [27].

Prior to building the ensemble model, we tested to determine the optimal architecture, focusing on different layer units and model depths. We performed a grid search over layer units of 64, 96, and 128, while varying the number of layers: the Dense layer ranged from 1 to

5, and the ConvBlock and GRU layers ranged from 1 to 3. Each configuration was evaluated on a validation set using accuracy. Considering the nature of the distributed system, we set upper limits for model parameters to ensure lightweight and computationally efficient models. In a distributed IoT environment, where nodes often have limited computational resources and energy, it is critical to minimize the number of model parameters to ensure efficient execution. While setting upper limits on parameters reduces model complexity and computational overhead, it may also constrain the model's ability to capture complex patterns in the data. The final architecture was chosen to maximize detection accuracy while adhering to the computational and energy constraints inherent in IoT environments.

3.4. Ensemble algorithms

Ensemble learning is a technique that combines multiple models to enhance the stability and predictive performance of the overall model. The ideology behind ensemble learning is to learn several simple models and combine their output to produce a final decision, aiming to combine the strengths of all and minimize the variances and biases of individual models. Assuming any ensemble learning algorithm can be denoted with function G to utilize the prediction of N models, with predicted labels $\{y_1, y_2, \dots, y_n\}$, the final prediction of the ensemble learning model can be represented by (1):

$$y = G(y_1, y_2, \dots, y_n) \quad (1)$$

There are two categories of ensemble learning methods: sequential methods, where base learners are generated consecutively, such as stacking and boosting, and parallel methods, where base learners work in parallel, such as voting. Stacking and voting can enhance prediction accuracy via further training and simple aggregation rules, respectively, whereas boosting can decrease bias [28]. Among them, the advantage of voting in ensemble methods is that it uses a simple aggregation rule but can make the final prediction more robust and accurate. Efficiency is the main reason for us to select and investigate this approach.

3.4.1. Voting

In a parallel training setting, individual classifiers may develop distinct representations and assign varying labels. Voting mechanisms are particularly effective in ensemble learning as they aggregate the predictions of multiple classifiers, reducing the risk of individual model biases and errors. To enhance the overall classifier performance, a voting mechanism is employed, which combines predictions either by selecting the class with the highest number of votes (hard voting) or by averaging the weighted probabilities of all classifiers' predictions (soft voting). Hard voting is often used when the base classifiers are diverse and produce categorical outputs, while soft voting is preferred when the classifiers output probabilistic predictions, allowing for a more nuanced combination of results [29].

Assuming we have three classifiers: C_1 , C_2 , and C_3 , each classifier predicts the class label for a given input sample. In hard voting, each classifier 'votes' for a class label, and the final prediction is simply the label that receives the majority of votes, similar to a democratic decision-making process. Denoting the predicted labels as y_1 , y_2 , and y_3 , respectively, the final predicted class label using hard voting can be written as (2):

$$y_{\text{final}} = \arg \max_c \sum_{i=1 \text{ to } 3} I(y_i == c) \quad (2)$$

where the function I represents a Boolean indicator function as follows, with c as the corresponding true labels:

$$I(y_i = c) = \begin{cases} 1 & \text{if True} \\ 0 & \text{if False} \end{cases} \quad (3)$$

For each prediction, we sum up the indicator function values across all classifiers, and the predicted label that can maximize this sum is chosen as the final predicted class label. In soft voting, on the other hand, we consider the predicted probabilities computed by individual classifiers. Soft voting aggregates the probabilistic predictions of each classifier, allowing for a weighted average of their confidence levels, which can result in more accurate and reliable predictions. Denoting the predicted probabilities of class label c by classifier C_i as $p_i(c)$, w_i as corresponding weight, the final prediction by soft voting algorithm is as follows:

$$y_{\text{final}} = \arg \max_c \sum_{i=1 \text{ to } 3} w_i \times p_i(c) \quad (4)$$

where the predicted probabilities are summed up with corresponding weights. The predicted label that can maximize this sum is chosen as the final prediction. In our experiment, the highest weight 0.4 is assigned to the base model with the highest accuracy, and 0.3 is assigned to the other two models. These weights were determined based on the classifiers' validation accuracy, ensuring that the most reliable classifiers have a greater influence on the final decision. By assigning higher weights to more accurate classifiers, weighted voting ensures that the ensemble's final prediction is influenced more by the models that are most likely to be correct, improving overall performance.

3.4.2. Stacking

Stacking, an ensemble technique, goes beyond simple aggregation by training an additional model to blend and refine the predictions from constituent classifiers. This final predictor, functioning as both a blender and a classifier, takes the ensemble's predictions as inputs to learn a composite representation and generate the final decision. Also referred to as 'stacked generalisation,' this method employs a more sophisticated strategy by integrating all base classifier outputs into a new model designed to optimize the combination of their predictions. We implement a deep DNN model as this final predictor, and the implementation and training details are presented in Algorithm 1. Initially, different base models (dnn_base, cnn_base, gru_base) are pre-trained individually for 30 epochs with the original datasets to learn prior knowledge. After training the base models, predictions are made on the training and validation data using each of the base models separately. These predictions are then horizontally stacked to create the input features, both for training and validation, for the stacking model. The stacking model contains DNN layers with 64 units, the GeLU activation function, a Dropout layer with a rate of 0.5, and the classification output layer with SoftMax activation. The stacking model is then compiled and trained for 50 epochs.

3.4.3. Boosting

Boosting is an ensemble technique that combines multiple weak learners into a single, stronger model by sequentially training them to refine each other's predictions. This study employs the Adaptive Boosting (AdaBoost) algorithm, as introduced in [30]. Boosting works by sequentially training weak learners, where each learner focuses on correcting the errors

Algorithm 1 Stacking_Implementation

```

1: Define base classifier  $models = [DNN, CNN, GRU]$ 
2: for  $model \in models$  do
3:   Pre-train the base classifier  $model.fit(X, y)$ 
4: end for
5:  $X\_stacked = horizontal\_stacking[prediction_{DNN}, prediction_{CNN}, prediction_{GRU}]$ 
6: Define the final predictor  $stacking\_model$  with input shape of  $n(labels) * 3$  and output
   shape of  $n(labels)$ 
7: Compile and train  $stacking\_model$ 

```

of its predecessor, ultimately combining their predictions into a single, more accurate model. AdaBoost achieves this by assigning weights to training instances, with misclassified instances receiving higher weights in subsequent iterations, ensuring that the model focuses more on difficult cases.

The algorithm's key strategy is to concentrate on underfitted instances, directing subsequent models to address those challenging cases. The weight of each instance is updated using the formula: $w_i^{(t+1)} = w_i^{(t)} \cdot \exp(\alpha_t \cdot I(y_i \neq h_t(x_i)))$, where α_t is the weight of the weak learner at iteration t , h_t is the weak learner's prediction, and I is an indicator function. The weight of each weak learner α_t is computed as $\alpha_t = \frac{1}{2} \ln(\frac{1-\epsilon_t}{\epsilon_t})$, where ϵ_t is the error rate of the weak learner at iteration t . Instances with higher weights are more likely to be selected for training in the next iteration, forcing the model to focus on previously misclassified cases.

In the boosting algorithm, only one base model should be used as the weak learner, and the success of boosting is significantly influenced by this choice. Given that CNN classifiers outperform others in one of our datasets and GRU classifiers show superior performance in the other, we adapt our approach by selecting the best-performing model for each dataset as the weak learner for AdaBoost. For the dataset where CNN excels, we pre-train a CNN for 30 epochs, ensuring the weak learner starts with a strong initial accuracy, allowing the boosting process to focus on refining predictions rather than correcting fundamental errors. Similarly, for the dataset where GRU performs best, we pre-train a GRU under the same conditions. In both cases, we implement ten weak learners based on the respective best model (CNN or GRU) in the AdaBoost process. The multi-class exponential loss function is employed during training, and the final boosted model combines the predictions of the weak learners using a weighted sum, where the weight of each learner is determined by its performance during training.

4. Experiments

4.1. Dataset

We utilize two public real-world datasets. The first one is CICIDS2017 [10], which can effectively evaluate intrusion detection and network traffic classification techniques. It comprises weekly users' behaviors across various protocols, including HTTP, HTTPS, FTP, SSH, and email. Table 1 presents the traffic types, distribution of the original, and after applying

Table 1. Data summary of CICIDS2017.

Traffic type	Original share	Share after resampling
BENIGN	84.3600%	12.5%
Bot	0.0800%	12.5%
Brute force	0.5400%	12.5%
DDoS	5.0300%	12.5%
DoS	9.9000%	12.5%
Heartbleed	0.0004%	12.5%
Infiltration	0.0014%	12.5%
Web attack	0.0900%	12.5%

Table 2. Data Summary of Edge-IIoT.

Traffic type	Original share (%)	Share after resampling (%)
Normal	72.830	54.364
DDoS_UDP	5.480	6.609
DDoS_ICMP	5.249	6.330
SQL_injection	2.308	2.784
Password	2.261	2.727
Vulnerability_scanner	2.259	2.724
DDoS_TCP	2.257	2.722
DDoS_HTTP	2.250	2.713
Uploading	1.696	2.718
Backdoor	1.121	2.718
Port_Scanning	1.017	2.718
XSS	0.717	2.718
Ransomware	0.492	2.718
Fingerprinting	0.045	2.718
MITM	0.018	2.718

resampling techniques. Rare but severe attacks like web attack, heartbleed and bot, only have an tiny share originally but are obviously enhanced after the resampling process.

The second dataset is Edge-IIoT [31]. The dataset contains over 2.2 million traffic samples and 61 correlated features spanning various network and application layers. As shown in Table 2, it encompasses 14 distinct types of attacks alongside normal behaviors, with normal traffic making up the majority (72.83%) of the dataset. This imbalanced distribution makes Edge-IIoT particularly suitable for evaluating the effectiveness of resampling techniques in handling class imbalance, as well as assessing the performance and efficiency of IDS in real-world IoT environments. Given that normal traffic constitutes a significant portion of the dataset and likely exhibits high diversity, we employ adaptive undersampling to reduce its volume while ensuring that approximately 50% of the original dataset is retained after the process.

4.2. Experiment setup

The dataset is split into 80% training and 20% validation with stratification on target labels. Stratification ensures that the training and validation sets maintain the same class distribution as the original dataset, preventing bias toward any specific class during training. To assess the superior performance of ensemble learning compared to DL models, we performed separate experiments with each base classifier for a comparative analysis. The comparative analysis aims to demonstrate the advantages of ensemble learning, such as improved robustness and accuracy, over traditional deep learning models, particularly in

Table 3. Overall multi-classification performance of CICIDS2017.

	Model	Training(%)				Validation(%)			
		Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1
Baseline	DNN	98.97	98.99	98.95	98.97	96.76	96.82	96.72	96.77
	CNN	99.19	99.20	99.18	99.19	98.05	98.08	97.98	98.03
	GRU	97.75	97.88	97.65	97.76	93.88	94.14	93.68	93.91
Ensemble	Hard-voting	99.16	99.17	99.16	99.16	97.41	98.63	97.41	97.92
	Soft-voting	99.18	99.20	99.18	99.18	97.49	98.65	97.49	97.98
	Stacking	99.35	99.35	99.34	99.35	98.29	98.30	98.29	98.29
	Boosting	99.24	99.24	99.24	99.24	98.31	98.32	98.32	98.32

handling complex and imbalanced datasets. All models employed the Adam optimizer with the following initialization: a learning rate of 0.001 and the categorical cross-entropy loss function. During the training, we utilize a learning rate scheduler to avoid plateau, namely, when the validation loss hasn't been improved for 3 epochs, the learning rate will decrease by a factor of 0.5. The learning rate scheduler prevents the model from getting stuck in local minima during training, by dynamically adjusting the learning rate based on the validation loss. They were trained for 50 epochs and a batch size of 1024, except for stacking and boosting algorithms, where the base models were pre-trained for an additional 30 epochs prior to the main training process. Pre-training helps initialize the weak learners with lower bias, which is critical so that boosting and stacking algorithms can focus on refining their predictions rather than correcting fundamental errors.

4.3. Experiment results

This section begins by examining the overall performance of individual models and ensemble algorithms. Subsequently, it delves into the classification performance of each ensemble algorithm, with a particular emphasis on rare attack detection.

Table 3 exhibits the performance of individual classifiers and ensemble models on CICIDS2017. Among the baseline models, the CNN classifier stands out on both training and validation sets; specifically, the accuracy of 98.05% and the F1 score of 98.03% demonstrates its superior performance. In contrast, the GRU model exhibits the weakest performance, with a validation accuracy of 93.88% and a validation F1-score of 93.91%. A noteworthy observation is that the CNN classifier exhibits minimal differences between training and validation results, suggesting better generalization, whereas the GRU model displays a tendency towards overfitting.

Ensemble methods generally exhibit enhanced performance compared to individual classifiers, highlighting the benefits of model combinations for improved overall results. Hard-voting and soft-voting surpass DNN and GRU models, but their superiority over CNN is limited due to slightly lower accuracy and F1 scores. This is because voting methods, while leveraging individual models' strengths, may not consistently outperform the best single model when the models possess diverse capabilities, as the aggregation rules are rather simple. The CNN model's excellence in feature extraction and nonlinear modeling provides a substantial advantage over simpler ensemble techniques. Among ensemble methods, boosting demonstrates the best validation performance without signs of overfitting, indicating its effectiveness in enhancing the predictive prowess of the base classifiers. This is

Table 4. Overall multi-classification performance of Edge-IIoT.

	Model	Training(%)				Validation(%)			
		Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1
Baseline	DNN	99.79	99.58	98.48	99.03	98.48	98.61	98.41	98.51
	CNN	99.81	99.62	98.02	98.81	98.02	98.36	97.76	98.06
	GRU	99.85	99.64	99.45	99.54	99.45	99.57	99.38	99.48
Ensemble	Hard-voting	99.76	99.77	99.76	99.76	99.54	99.61	99.54	99.56
	Soft-voting	99.76	99.77	99.76	99.76	99.69	99.61	99.60	99.63
	Stacking	99.79	99.79	99.79	99.79	99.75	99.78	99.75	99.76
	Boosting	99.69	99.61	99.81	99.79	99.76	99.69	99.79	99.76

partly attributed to the inherent strength of the CNN classifier. Stacking also shows strong performance on the validation set and the best performance on the training set, with an exceptional accuracy of 99.35% and an F1 score of 99.35%, indicating that the meta-learner adeptly combines base models' predictions. Notably, despite GRU's under-performance as an individual classifier, stacking enabled it to achieve high accuracy and F1 scores even when weaker models were included, demonstrating the potential of ensemble algorithms to compensate for weaker base models.

Table 4 exhibits the performance of individual classifiers and ensemble models built upon them on Edge-IIoT. The evaluation metrics of the base models indicate strong individual performance, with all three models achieving high accuracy, precision, recall, and F1 scores. The DNN shows consistent performance with training accuracy of 99.79% and validation accuracy of 99.03%, maintaining high precision (98.48%) and recall (98.41%). The CNN performs slightly better in training accuracy at 99.81%, but its validation accuracy is marginally lower at 98.81%, with a slight dip in precision and recall. The GRU outperforms both in training and validation, achieving the highest training accuracy (99.85%) and validation accuracy (99.48%), along with the best precision (99.45%) and recall (99.47%). Overall, the GRU demonstrates the most robust performance across all metrics, followed closely by the DNN and CNN. These results suggest that while each model performs well independently, ensemble learning could potentially combine their strengths to further enhance accuracy and robustness.

Among the ensemble algorithms, hard-voting achieves a slightly higher validation accuracy (99.54%) compared to the base models, with precision and recall both at 99.54%. Soft-voting shows a validation accuracy of 99.69%, which is marginally better than hard-voting and the base models. Precision and recall are also slightly higher. Using majority voting to combine the predictions of the base models seems to slightly improve overall performance. Weighted voting, which is based on prediction probabilities, can lead to more accurate predictions. The stacking ensemble achieves a validation accuracy of 99.75%, the highest precision and F1, demonstrating that stacking, which uses a meta-model to combine predictions, can significantly enhance performance. Boosting achieves the highest validation accuracy of 99.76% and superior recall and F1, indicating that boosting, which focuses on correcting errors from previous models, is effective in improving generalization.

We would like to further investigate the detection capabilities of ensemble learning approaches, particularly their effectiveness with minority classes. The classification reports on Edge-IIoT are presented in Table 5. Given that the classification of benign behaviors and DoS and DDoS attacks is quite effective, we will focus on those rare attacks with an original

Table 5. Classification report of ensemble models of CICIDS2017.

Type	Stacking			Hard-voting			Soft-voting			Boosting		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
BENIGN	1	0.98	0.99	1	0.97	0.98	1	0.97	0.98	1	0.98	0.99
Bot	0.12	0.99	0.21	0.08	1	0.14	0.08	1	0.15	0.12	0.99	0.22
Brute Force	0.88	0.99	0.94	0.86	1	0.93	0.85	1	0.92	0.88	0.97	0.92
DDoS	0.96	1	0.98	0.95	1	0.98	0.95	1	0.98	0.96	1	0.98
DoS	0.95	1	0.97	0.91	1	0.95	0.91	1	0.95	0.95	1	0.97
Heartbleed	1	1	1	1	1	1	0.67	1	0.8	1	0.5	0.67
Infiltration	0.05	0.71	0.08	0.16	0.71	0.26	0.14	0.71	0.23	0.11	0.57	0.18
Web Attack	0.22	0.9	0.36	0.2	0.9	0.32	0.19	0.9	0.32	0.22	0.94	0.36

Table 6. Classification report of ensemble models of Edge-IIoT.

Type	Hard-voting			Soft-voting			Stacking			Boosting		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
Backdoor	1	0.97	0.98	1	0.97	0.98	1	0.97	0.98	1	0.97	0.98
DDoS_HTTP	0.99	1	0.99	0.99	1	0.99	0.99	0.99	0.99	0.99	1	0.99
DDoS_ICMP	0.99	1	1	1	1	1	0.99	1	1	1	1	1
DDoS_TCP	0.99	1	1	0.99	0.99	0.99	1	1	1	1	1	1
DDoS_UDP	1	1	1	1	1	1	1	1	1	1	1.00	1
Fingerprinting	0.96	0.75	0.84	0.96	0.75	0.84	0.96	0.77	0.85	0.97	0.75	0.85
MITM	0.32	1	0.48	1	0.99	0.99	0.34	1	0.5	0.23	1	0.37
Normal	1	1	1	1	1	1	1	1	1	1	1	1
Password	1	0.98	0.99	0.99	0.98	0.98	0.99	0.99	0.99	1	0.98	0.99
Port_Scanning	0.81	1	0.89	0.91	1	0.95	0.92	1	0.95	0.81	1	0.9
Ransomware	0.98	0.89	0.93	0.98	0.89	0.93	0.98	0.89	0.93	0.98	0.89	0.93
SQL_injection	0.99	1	0.99	0.98	1	0.99	0.99	1	0.99	0.99	1	0.99
Uploading	0.97	1	0.98	0.99	1	0.99	1	1	1	0.99	1	0.99
Vulnerability_scanner	1	0.99	1	0.99	0.99	0.99	0.99	1	0.99	1	0.99	0.99
XSS	0.99	1	0.99	0.97	0.99	0.98	0.99	1	0.99	0.99	1	0.99
Backdoor	1	0.97	0.98	1	0.97	0.98	1	0.97	0.98	1	0.97	0.98

share lower than 1%. All models, except boosting, have a recall of 1 for Heartbleed, indicating excellent detection, while the precision is also acceptable. For Brute Force attacks, voting algorithms deliver the best detection as they achieve a recall of 1, whereas stacking and boosting exhibit more balanced performance, with F1 scores of 94% and 92%, respectively. For the other three rare attacks, namely Bot, Infiltration, and Web Attack, although the recall scores are generally high across all ensemble models, the low precision and F1 scores highlight areas for improvement. Nevertheless, the sensitivity to anomalies is crucial for IDS, as the consequence of incorrectly labeling attacks as normal behavior is more severe than raising a false alarm, which prioritizes high recall over low false alarms in this evaluation.

Table 6 presents the classification report on Edge-IIoT dataset. All Ensemble Methods achieve perfect precision, recall, and F1 scores (100%) for normal traffic. This indicates that the majority class is consistently and accurately classified across all methods. For the rare attacks, the performance varies depending on the specific attack type and the ensemble method used. For Backdoor, all ensemble methods achieve perfect precision (100%) and high recall ($\sim 97\%$), leading to strong F1 scores (98%). This suggests that Backdoor attacks are well-classified across all ensemble techniques. For Port_Scanning, Ransomware and XSS, all ensemble methods have fair performance. For Fingerprinting, all methods have high precision (96–97%) but low recall ($\sim 75\text{--}77\%$). This indicates that while the models

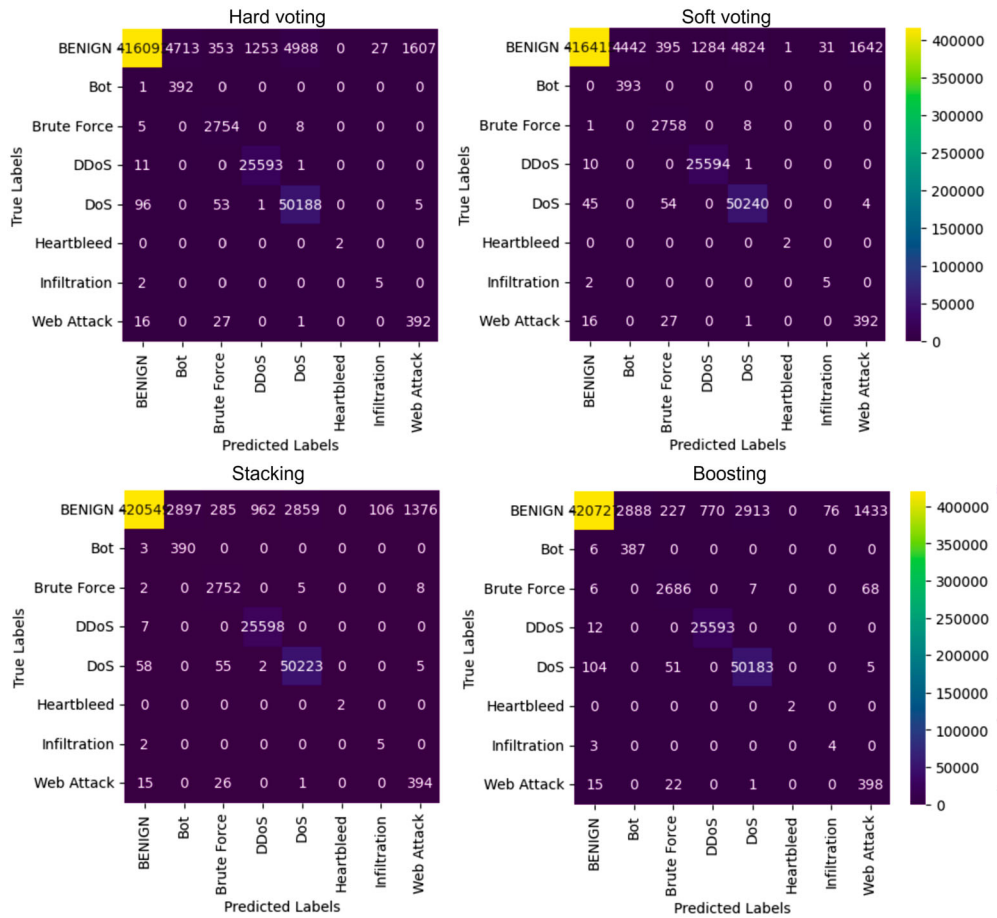


Figure 4. Confusion metrics of ensemble classifiers – CICIDS2017.

are precise in identifying Fingerprinting attacks, they miss a significant portion of these attacks. For MITM (Man-in-the-Middle), hard-voting has low precision (32%) but perfect recall (100%), leading to a low F1 score (48%); Stacking and Boosting also have similar observation. This means it is prone to false positives when detecting MITM attacks. Soft-voting shows perfect precision and recall (100% and 99%, respectively), which performs the best. Except for a few exceptions such as fingerprinting and MITM, all ensemble approaches excel in classifying regular traffic and the rarest attacks. Still, all models can detect a portion of the attacks, even in challenging cases.

The confusion metrics of all ensemble learning approaches are presented in Figure 4. Boosting exhibits the lowest false positive rate for corrected benign samples and demonstrates superior performance in detecting major attacks like DoS and DDoS with low false alarms. However, it stands out with the highest false alarms for the infiltration class compared to the other three ensemble methods. This disparity can be attributed to boosting's reliance on a single weak classifier, CNN, in contrast to ensemble algorithms that consider all base classifiers. Notably, despite the Heartbleed attack's limited presence (two cases in the validation set), all models demonstrated effective detection without excessive false alarms.

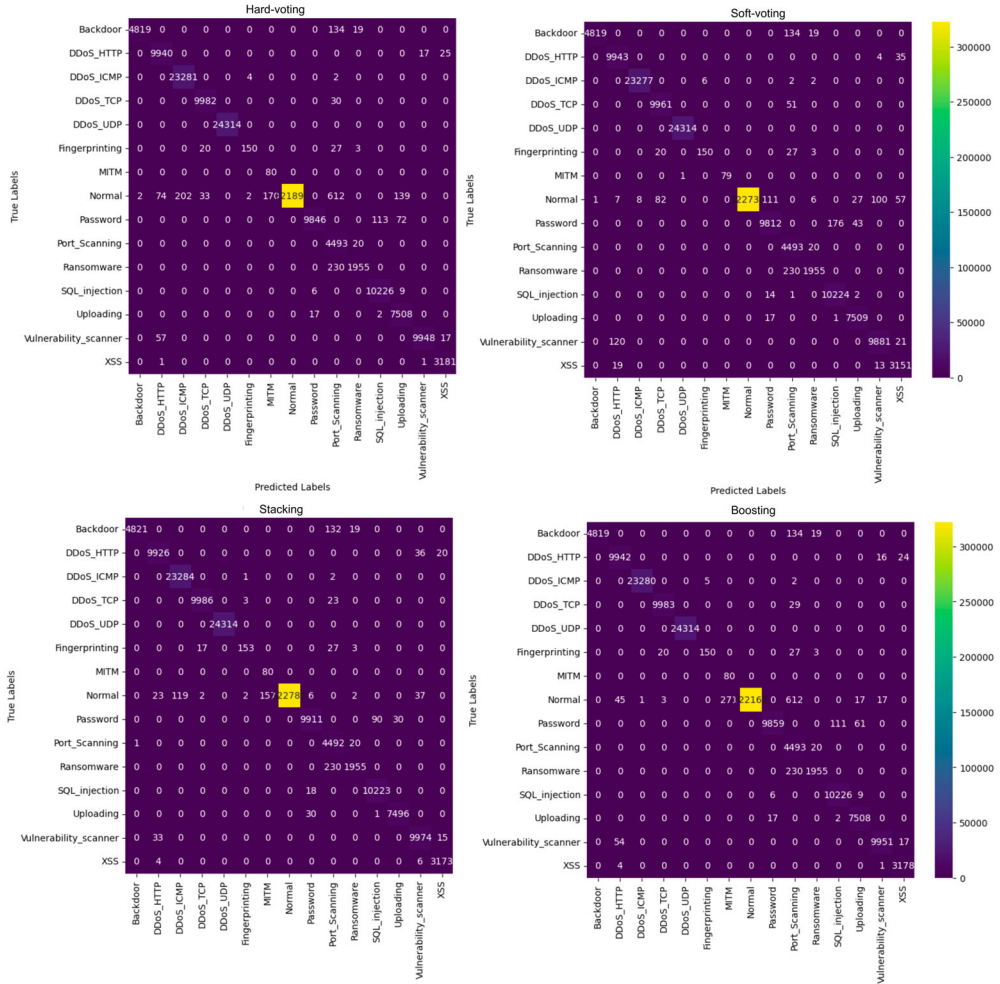


Figure 5. Confusion metrics of ensemble classifiers – Edge-IloT.

This strong performance on minority classes validates the efficacy of our hybrid undersampling method in addressing class imbalances within the training data. With misclassified benign cases, the overall false alarm rates for benign behaviors can be calculated. In the test set, the percentage of wrongly classified benign behaviors is as follows: hard voting – 3.02%, soft voting – 2.94%, stacking – 1.98%, and boosting – 1.94%. Too many false alarms can significantly affect normal operations; to minimize potential distractions and operational disruptions, stacking and boosting are deemed more appropriate as intrusion detection techniques.

The confusion metrics of all ensembles learning approaches on Edge-IloT are presented in Figure 5, using the confusion matrix. Similarly, most cases are classified correctly. For misclassified benign cases, the overall false alarm rates for benign behaviors can be calculated. In the test set, the cases and percentage of wrongly classified benign behaviors is as follows: hard voting – 1234 (0.382%), soft voting – 399 (0.123%), stacking – 348 (0.108%), and boosting – 965 (0.299%). Stacking has the lowest false alarm rate, which indicates that Stacking is

the most effective at correctly classifying benign traffic, minimizing the risk of false alarms. This is particularly important in environments where false positives can lead to unnecessary disruptions.

Overall, the ensemble methods, particularly Stacking and Boosting, demonstrate superior performance compared to the individual base models (DNN, CNN, GRU). Both methods exhibit competitive accuracy and F1-scores, with Stacking achieving slightly higher precision and Boosting showing marginally superior recall. While their performance is nearly comparable, Stacking emerges as the preferred choice due to its robustness, which stems from its integration of multiple base classifiers within a dynamic model architecture. This approach allows Stacking to effectively combine the strengths of diverse models, leading to more reliable and accurate predictions across varied scenarios. Thus, Stacking stands out as the optimal solution for this dataset, offering both high performance and enhanced generalizability.

4.4. Discussion on complexity and usability

Firstly, we analyze the complexity of base models. The DNN's complexity is $O(n \times m)$, where n is the input dimension and m is the number of units in each dense layer. In our case, each DNN layer contains 128 units. The CNN consists of 3 blocks, each containing a 1D convolutional layer with 64 filters, batch normalization, and max-pooling. The CNN's complexity is primarily driven by the convolutional operations, which are $O(k \times m \times n)$, where k is the kernel size 3, m is the number of filters (64), and n is the input dimension. The GRU consists of 3 blocks, each containing a GRU layer with 64 units, followed by a dropout layer. The GRU's complexity is $O(m^2 \times n)$, where m is the number of units (64) and n is the input dimension. GRUs are inherently more computationally expensive than DNNs and CNNs due to the recurrent operations.

In terms of computational complexity, the DNN is the most efficient model, making it suitable for tasks where computational resources are limited. The CNN is more complex due to its convolutional layers but is efficient for spatial data. The GRU, while powerful for sequential data, is the most computationally expensive due to its recurrent operations.

IDS are among the most critical technologies for safeguarding IoT ecosystems, necessitating the adoption of advanced techniques that enhance predictive accuracy and generalizability. While the successful deployment of distributed IDSs in resource-constrained environments is paramount, such systems must also maintain high performance within the limitations of distributed computational resources. To address this dual challenge, deep learning (DL) models must be specifically optimized to achieve a balance between real-time responsiveness and detection accuracy across distributed nodes, even under stringent computational constraints.

The deployment of an effective IDS in IoT environments necessitates a careful balance between predictive accuracy, computational efficiency, and usability [32]. The design is rooted in a resource-constrained framework, ensuring compatibility with a spectrum of devices ranging from low-power edge nodes to more robust fog and cloud servers. The base models (DNN, CNN, GRU) are specifically optimized to reduce memory usage and computational overhead. For example, the DNN's dense layers, configured with default parameters (128 units, 5 blocks), require only ~ 0.5 –2 MB of memory, making them suitable for deployment on devices with limited RAM. Furthermore, the inclusion of three robust

base models enables adaptive model selection based on varying computational capacities. In scenarios where GPU resources are unavailable, the GRU model can be seamlessly replaced by either the DNN or CNN, as they exhibit comparable performance across diverse datasets.

For usability and practical integration, the framework leverages pre-trained models that support containerized deployment (e.g. Docker/Kubernetes), ensuring seamless integration into existing IoT architectures. It is also compatible with leading edge computing platforms such as AWS Greengrass and Azure IoT Edge. However, certain critical factors, including security and compliance, were not addressed in the proposed work. To address data privacy concerns, federated learning can be incorporated, enabling nodes to collaboratively refine detection models without the need to share raw data [33]. This approach not only enhances privacy but also aligns with regulatory requirements such as GDPR and HIPAA.

5. Conclusion

In this research, we conducted comparative analysis for ensemble DL learning techniques, with both individual classifiers and ensemble algorithms examined. In general, the experiment results suggested that ensemble methods like stacking and boosting can provide significant improvements in classification accuracy and other performance metrics compared to individual classifiers. Furthermore, the adaptability of selected ensemble models facilitates their application in distributed IDS setups, where multiple, lightweight DL models could collaboratively function. This capacity for collaboration without the need for intensive computational resources makes our approach particularly suited for distributed environments.

While the proposed ensemble learning framework demonstrates robust performance in intrusion detection for IoT environments, several improvement and practical deployment considerations remain unexplored. With the focus on stacking and boosting algorithm, the following plan will be carried out. First, future work will focus on scalability and system integration, particularly refining the framework's compatibility with lightweight IoT. This includes optimizing structure and substituting GRU base model with simpler, CPU-friendly ones. Second, real-time usability in operational scenarios will be prioritized by developing adaptive learning algorithms, setting resource thresholds like minimum RAM (< 512 MB) and CPU utilization ($< 50\%$). Further research will also explore hybrid architectures that fuse rule-based filtering with ensemble predictions to reduce false alarms in high-noise environments.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

The reported study was partially funded by the budget project [FFZF-2025-0016]; Russian Academy of Sciences.

ORCID

Huiyao Dong  <http://orcid.org/0009-0008-7371-4908>

Igor Kotenko  <http://orcid.org/0000-0001-6859-7120>

References

- [1] IoT Business News. State of IoT 2024: number of connected IoT devices growing 13% to 18.8 billion globally. 2024 [accessed 2024 September 4]. Available from: <https://iotbusinessnews.com/2024/09/04/26399-state-of-iot-2024-number-of-connected-iot-devices-growing-13-to-18-8-billion-globally>
- [2] Sun P, Wan Y, Wu Z, et al. A survey on privacy and security issues in IoT-based environments: technologies, protection measures and future directions. *Comput Secur.* **2025**;148:104097. doi: [10.1016/j.cose.2024.104097](https://doi.org/10.1016/j.cose.2024.104097)
- [3] Dong H, Kotenko I. Multi-task learning for IoT traffic classification: a comparative analysis of deep autoencoders. *Future Gener Comput Syst.* **2024**;158:242–254. doi: [10.1016/j.future.2024.04.005](https://doi.org/10.1016/j.future.2024.04.005)
- [4] Branitskiy A, Kotenko I. Network attack detection based on combination of neural, immune and neuro-fuzzy classifiers. In: 2015 IEEE 18th International Conference on Computational Science and Engineering, Porto, Portugal; **2015**. p. 152–159. doi: [10.1109/CSE.2015.26](https://doi.org/10.1109/CSE.2015.26)
- [5] Branitskiy A, Kotenko I, Saenko I. Applying machine learning and parallel data processing for attack detection in IoT. *IEEE Trans Emerg Top Comput.* **2020**;9(4):1642–1653. doi: [10.1109/TETC.2020.3006351](https://doi.org/10.1109/TETC.2020.3006351)
- [6] Sanju P. Enhancing intrusion detection in IoT systems: a hybrid metaheuristics-deep learning approach with ensemble of recurrent neural networks. *J Eng Res.* **2023**;11(4):356–361. doi: [10.1016/j.jer.2023.100122](https://doi.org/10.1016/j.jer.2023.100122)
- [7] Alem S, Espes D, Nana L. A novel bi-anomaly-based intrusion detection system approach for industry 4.0. *Future Gener Comput Syst.* **2023**;145:267–283.
- [8] Idrissi MJ, Alami H, El Mahdaouy A, et al. Fed-ANIDS: federated learning for anomaly-based network intrusion detection systems. *Expert Syst Appl.* **2023**;234:121000. doi: [10.1016/j.eswa.2023.121000](https://doi.org/10.1016/j.eswa.2023.121000)
- [9] de Carvalho Bertoli G, Alves Pereira Junior L, Saotome O, et al. Generalizing intrusion detection for heterogeneous networks: a stacked-unsupervised federated learning approach. *Comput Secur.* **2023**;127:103106. doi: [10.1016/j.cose.2023.103106](https://doi.org/10.1016/j.cose.2023.103106)
- [10] Sharafaldin I, Habibi Lashkari A, Ghorbani AA. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy – ICISPP; INSTICC*. SciTePress; 2018. p. 108–116.
- [11] Dong H, Kotenko I. Enhancing IoT security with ensemble deep learning: a lightweight, robust approach to intrusion detection. In: *Advancements in Distributed AI Systems Intelligent Distributed Computing XVII: 17th International Symposium on Intelligent Distributed Computing*; University of Brighton, United Kingdom. doi: [10.1007/978-3-031-87639-4_5](https://doi.org/10.1007/978-3-031-87639-4_5)
- [12] Tama BA, Lim S. Ensemble learning for intrusion detection systems: a systematic mapping study and cross-benchmark evaluation. *Comput Sci Rev.* **2021**;39:100357. doi: [10.1016/j.cosrev.2020.100357](https://doi.org/10.1016/j.cosrev.2020.100357)
- [13] Fitni QRS, Ramli K. Implementation of ensemble learning and feature selection for performance improvements in anomaly-based intrusion detection systems. In: *2020 IEEE International Conference on Industry 4.0, Artificial Intelligence, and Communications Technology (IAICT)*; Bali, Indonesia; **2020**. p. 118–124. doi: [10.1109/IAICT50021.2020.9172014](https://doi.org/10.1109/IAICT50021.2020.9172014)
- [14] Abirami MS, Yash U, Singh S. Building an ensemble learning based algorithm for improving intrusion detection system. In: *SS Dash, C Lakshmi, S Das, et al., editors. Artificial intelligence and evolutionary computations in engineering systems*. Singapore: Springer; **2020**. p. 635–649.
- [15] Gao X, Shan C, Hu C, et al. An adaptive ensemble machine learning model for intrusion detection. *IEEE Access.* **2019**;7:82512–82521. doi: [10.1109/Access.6287639](https://doi.org/10.1109/Access.6287639)

- [16] ud Din SM, Sharma R, Rizvi F, et al. Detection of botnet in IoT network through machine learning based optimized feature importance via ensemble models. *Int J Inf Technol.* **2023**;1–9. Available from: <https://api.semanticscholar.org/CorpusID:265299711>
- [17] Kumar R, Kumar P, Tripathi R, et al. A distributed intrusion detection system to detect DDoS attacks in blockchain-enabled IoT network. *J Parallel Distrib Comput.* **2022**;164:55–68. doi: [10.1016/j.jpdc.2022.01.030](https://doi.org/10.1016/j.jpdc.2022.01.030)
- [18] Kumar P, Gupta GP, Tripathi R. Design of anomaly-based intrusion detection system using fog computing for IoT network. *Autom Control Comput Sci.* **2021**;55(2):137–147. doi: [10.3103/S0146411621020085](https://doi.org/10.3103/S0146411621020085)
- [19] Sharma R, Chan CA, Leckie C. Probabilistic distributed intrusion detection for zero-trust multi-access edge computing. In: *NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium*. Miami, FL: IEEE; **2023**. p. 1–9. doi: [10.1109/NOMS56928.2023.10154326](https://doi.org/10.1109/NOMS56928.2023.10154326)
- [20] Hnamte V, Hussain J. DCNNBiLSTM: an efficient hybrid deep learning-based intrusion detection system. *Telemat Inform Rep.* **2023**;10:100053. doi: [10.1016/j.teler.2023.100053](https://doi.org/10.1016/j.teler.2023.100053)
- [21] Khonde SR, Ulagamuthalvi V. Blockchain: secured solution for signature transfer in distributed intrusion detection system. *Comput Syst Sci Eng.* **2022**;40(1):37–51.
- [22] Salih Hasan BM, Abdulazeez AM. A review of principal component analysis algorithm for dimensionality reduction. *J Soft Comput Data Min.* **2021 Apr**;2(1):20–30.
- [23] Chawla N, Bowyer K, Hall L, et al. SMOTE: synthetic minority over-sampling technique. *J Artif Intell Res (JAIR).* **2002 Jun**;16:321–357. doi: [10.1613/jair.953](https://doi.org/10.1613/jair.953)
- [24] Sesmero MP, Iglesias JA, Magán E, et al. Impact of the learners diversity and combination method on the generation of heterogeneous classifier ensembles. *Appl Soft Comput.* **2021**;111:107689. doi: [10.1016/j.asoc.2021.107689](https://doi.org/10.1016/j.asoc.2021.107689)
- [25] Park S, Kwak N. Analysis on the dropout effect in convolutional neural networks. In: *Computer Vision–ACCV 2016: 13th Asian Conference on Computer Vision*; Taipei, Taiwan; November 20–24, 2016; Revised Selected Papers, Part II 13. Springer; 2017. p. 189–204.
- [26] Mohammadpour L, Ling TC, Liew CS, et al. A survey of CNN-based network intrusion detection. *Appl Sci.* **2022**;12(16):8162. doi: [10.3390/app12168162](https://doi.org/10.3390/app12168162)
- [27] Bajao NA, Sarucam J-a. Threats detection in the internet of things using convolutional neural networks, long short-term memory, and gated recurrent units. *Mesop J Cybersecur.* **2023**;2023:22–29. doi: [10.58496/MJCS/2023/005](https://doi.org/10.58496/MJCS/2023/005)
- [28] Mohammed A, Kora R. A comprehensive review on ensemble deep learning: opportunities and challenges. *J King Saud Univ Comput Inform Sci.* **2023 Feb**;35:757–774.
- [29] Mienye ID, Sun Y. A survey of ensemble learning: concepts, algorithms, applications, and prospects. *IEEE Access.* **2022**;10:99129–99149. doi: [10.1109/ACCESS.2022.3207287](https://doi.org/10.1109/ACCESS.2022.3207287)
- [30] Zhu J, Rosset S, Zou H. Multi-class adaboost. *Stat Interface.* **2006 Feb**;2:349–360.
- [31] Ferrag MA, Friha O, Hamouda D, et al. Edge-IIoTset: a new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning. *IEEE Access.* **2022**;10:40281–40306. doi: [10.1109/ACCESS.2022.3165809](https://doi.org/10.1109/ACCESS.2022.3165809)
- [32] Khraisat A, Alazab A. A critical review of intrusion detection systems in the internet of things: techniques, deployment strategy, validation strategy, attacks, public datasets and challenges. *Cybersecurity.* **2021**;4:1–27. doi: [10.1186/s42400-021-00077-7](https://doi.org/10.1186/s42400-021-00077-7)
- [33] Campos EM, Saura PF, González-Vidal A, et al. Evaluating federated learning for intrusion detection in internet of things: review and challenges. *Comput Netw.* **2022**;203:108661. doi: [10.1016/j.comnet.2021.108661](https://doi.org/10.1016/j.comnet.2021.108661)